

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA INGENIERIA Y DISEÑO
DIGITAL

Carrera de Software



ANÁLISIS CRÍTICO DE KUBERNETES
COMO SISTEMA DISTRIBUIDO

Comunicación, Tolerancia a Fallos y Consistencia Distribuida



ASIGNATURA

Aplicaciones Distribuidas

ELABORADO POR:

Freddy Vladimir Farinango Guandinango
Séptimo Semestre Software "A"

DOCENTE:

Ing. Gleiston Cicerón Guerrero Ulloa

2025

ANÁLISIS CRÍTICO DE KUBERNETES COMO SISTEMA DISTRIBUIDO

Comunicación, Tolerancia a Fallos y Consistencia Distribuida

Índice

1. Introducción	3
2. Fundamentos de Kubernetes como Sistema Distribuido	3
2.1. Descripción y características	3
2.2. Ventajas y desafíos	3
3. Modelos y Comunicación en Kubernetes	4
3.1. Modelo cliente-servidor y comunicación entre procesos	4
3.2. Middleware, coordinación y sincronización	4
4. Seguridad y Confiabilidad en Kubernetes	4
4.1. Amenazas, autenticación y cifrado	4
4.2. Tolerancia a fallos y alta disponibilidad	5
5. Coordinación y Consistencia Distribuida	5
5.1. Modelo CAP y algoritmos de coordinación	5
5.2. Consenso distribuido y rol de etcd	5
6. Análisis Crítico y Evaluación	6
7. Conclusiones	6
8. Bibliografía	6

1. Introducción

Los sistemas distribuidos representan una base fundamental de la computación moderna, ya que integran múltiples recursos que trabajan coordinadamente mediante redes de comunicación. En este contexto, Kubernetes se ha convertido en una herramienta ampliamente utilizada para la orquestación de contenedores en entornos distribuidos, automatizando el despliegue, escalado y recuperación de servicios en infraestructuras de nube.

El presente informe analiza críticamente a Kubernetes como sistema distribuido real, aplicando conceptos de la Unidad 1: transparencia, comunicación, tolerancia a fallos, seguridad y coordinación distribuida, evaluando sus ventajas, limitaciones y desafíos.

2. Fundamentos de Kubernetes como Sistema Distribuido

2.1. Descripción y características

Kubernetes es una plataforma de código abierto que automatiza el despliegue y administración de aplicaciones basadas en contenedores. Como sistema distribuido, presenta características como concurrencia, escalabilidad y transparencia de distribución, además de mecanismos para balancear tráfico y recuperar servicios ante fallos [1].

Desarrollado originalmente por Google, Kubernetes se ha convertido en una de las plataformas más utilizadas para la orquestación de contenedores en infraestructuras de nube. A diferencia de los sistemas centralizados y paralelos, distribuye tareas y servicios entre múltiples nodos independientes conectados mediante redes de comunicación, mejorando la disponibilidad y flexibilidad del sistema. Sin embargo, esta arquitectura también incrementa la complejidad de coordinación y administración de recursos.

2.2. Ventajas y desafíos

Categoría	Elemento	Descripción
Ventajas	Escalabilidad automática	Permite aumentar o reducir recursos automáticamente según la demanda.
	Alta disponibilidad	Mantiene servicios activos y distribuidos entre varios nodos.
	Tolerancia a fallos	Detecta errores y reinicia automáticamente los servicios afectados.
Desafíos	Complejidad administrativa	La configuración puede resultar compleja por la cantidad de componentes internos.
	Seguridad	La protección de contenedores y comunicaciones es un reto constante.
	Sincronización	La coordinación entre múltiples nodos puede generar dificultades a gran escala.

3. Modelos y Comunicación en Kubernetes

3.1. Modelo cliente-servidor y comunicación entre procesos

Kubernetes implementa principalmente un modelo cliente-servidor. El plano de control administra recursos y coordina los nodos, mientras que el API Server actúa como punto central de comunicación, recibiendo instrucciones de despliegue y administración. Los nodos trabajadores ejecutan cargas de trabajo y mantienen comunicación constante con el plano de control [2].

La comunicación se realiza mediante APIs REST y protocolos de red modernos, adaptados a infraestructuras de nube y microservicios. Los contenedores pueden comunicarse entre sí utilizando servicios internos que facilitan el descubrimiento y balanceo de tráfico dentro del clúster [1].

Aunque Kubernetes utiliza principalmente una arquitectura cliente-servidor, también presenta características híbridas debido a la cooperación y comunicación constante entre los diferentes nodos del clúster.

3.2. Middleware, coordinación y sincronización

El middleware en Kubernetes incluye componentes como el scheduler, el controller manager y etcd. El scheduler asigna contenedores a nodos según disponibilidad de recursos; etcd almacena la información crítica y el estado del sistema. Estos componentes mantienen sincronización constante entre nodos.

Aunque Kubernetes no implementa directamente el algoritmo de Lamport, utiliza principios de coordinación temporal y control de eventos distribuidos para mantener coherencia entre procesos y recursos compartidos, evitando inconsistencias y conflictos de actualización.

4. Seguridad y Confiabilidad en Kubernetes

4.1. Amenazas, autenticación y cifrado

Kubernetes enfrenta amenazas relacionadas con accesos no autorizados, ataques de red y configuraciones inseguras. Los principales riesgos incluyen permisos mal configurados, exposición insegura de APIs y contenedores comprometidos. Según [3], la ausencia de comunicación segura puede derivar en suplantación de identidad, violación de integridad de datos e indisponibilidad de servicios críticos.

Para contrarrestar estas amenazas, Kubernetes implementa RBAC (Role-Based Access Control) para asignar permisos según roles definidos, además de cifrado y protocolos seguros de comunicación mediante certificados digitales. En este sentido, [4] propone el uso del framework SPIFFE/SPIRE para asignar identidades criptográficas únicas a los servicios distribuidos, fortaleciendo la autenticación mutua entre microservicios en clústeres Kubernetes.

Complementariamente, [3] demuestra que las políticas de red basadas en eBPF, como las implementadas por Calico y Cilium, representan soluciones de seguridad de bajo impacto en el rendimiento, adecuadas para comunicaciones entre contenedores con requisitos de baja latencia.

4.2. Tolerancia a fallos y alta disponibilidad

La tolerancia a fallos es una de las características más importantes de Kubernetes. Cuando un contenedor presenta fallos, la plataforma puede reiniciarlo automáticamente o reemplazarlo por una nueva instancia. Si un nodo deja de responder, las cargas de trabajo se redistribuyen hacia otros nodos disponibles [1].

La alta disponibilidad se garantiza mediante réplicas, balanceo de carga y automatización de monitoreo y recuperación. [2] demuestra que un clúster Kubernetes en bare-metal es capaz de continuar operando ante la pérdida de nodos, redistribuyendo las tareas en cuestión de segundos sin intervención humana. Para escenarios de desastre total, [1] propone un sistema automatizado de backup y restauración mediante snapshots de etcd y AWS, reduciendo el tiempo de recuperación de 300 a 61 segundos en comparación con procesos manuales.

Respecto al monitoreo, [5] señala que el sistema de monitorización nativo de Kubernetes requiere intervención humana para detectar defectos, frecuentemente cuando ya es demasiado tarde. Por ello, propone una solución basada en Prometheus para la detección automática de anomalías, permitiendo anticipar fallos antes de que impacten en la disponibilidad del clúster.

5. Coordinación y Consistencia Distribuida

5.1. Modelo CAP y algoritmos de coordinación

El modelo CAP establece que un sistema distribuido no puede garantizar simultáneamente consistencia, disponibilidad y tolerancia a particiones. En Kubernetes, la disponibilidad y la tolerancia a particiones son fundamentales para mantener el funcionamiento continuo ante fallos de red, mientras que etcd permite mantener sincronización y consistencia entre los componentes del clúster.

En cuanto a coordinación, ciertos procesos de Kubernetes utilizan mecanismos de elección de líder para evitar conflictos y garantizar una administración ordenada. Aunque no implementa directamente algoritmos clásicos como Bully o Ring, aplica principios de coordinación distribuida para mantener estabilidad operativa.

5.2. Consenso distribuido y rol de etcd

El consenso distribuido es esencial en Kubernetes, ya que múltiples componentes deben compartir información consistente sobre el estado del clúster. Para ello, la plataforma utiliza etcd, una base de datos distribuida que implementa mecanismos de consenso basados en el algoritmo Raft, coordinando nodos y manteniendo la integridad de los datos ante posibles fallos [1].

En cuanto a transacciones, aunque Kubernetes no utiliza directamente el protocolo Two-Phase Commit (2PC), implementa procesos de verificación y sincronización que mantienen coherencia entre el estado deseado y el estado real de las aplicaciones, minimizando errores derivados de fallos parciales o interrupciones de red.

6. Análisis Crítico y Evaluación

Desde una perspectiva crítica, Kubernetes es una plataforma eficiente para administrar aplicaciones distribuidas modernas en entornos de nube y microservicios. Su automatización facilita tareas de despliegue, monitoreo y recuperación de servicios, permitiendo mantener continuidad operativa y administrar recursos según la demanda.

Sin embargo, presenta limitaciones relevantes. Su complejidad de implementación requiere conocimientos especializados en redes, contenedores, seguridad y coordinación distribuida, lo que puede dificultar su adopción en organizaciones con poca experiencia técnica. Errores de configuración pueden generar vulnerabilidades o interrupciones en los servicios.

Otro desafío importante es el consumo de recursos: Kubernetes necesita infraestructura adecuada para operar eficientemente. En proyectos pequeños, su implementación puede resultar innecesariamente compleja frente a soluciones más simples. Asimismo, la seguridad de las comunicaciones entre microservicios sigue siendo un reto activo [4], y la sincronización entre múltiples nodos puede generar dificultades relacionadas con latencia y consistencia de datos [5].

En términos generales, Kubernetes representa una solución sólida para arquitecturas distribuidas modernas, gracias a su flexibilidad, automatización y escalabilidad. No obstante, su implementación debe evaluarse según las necesidades reales de cada entorno tecnológico.

7. Conclusiones

El análisis permitió comprender la importancia de Kubernetes dentro de los sistemas distribuidos modernos. Su arquitectura basada en múltiples nodos demuestra la aplicación práctica de conceptos como escalabilidad, concurrencia, transparencia y tolerancia a fallos. Asimismo, componentes como etcd y el algoritmo Raft ilustran mecanismos reales de sincronización, seguridad y consistencia distribuida.

En conclusión, Kubernetes refleja principios fundamentales de los sistemas distribuidos y representa una herramienta clave para la administración de aplicaciones modernas en infraestructuras de computación actuales, aunque su complejidad exige una evaluación cuidadosa antes de su adopción.

8. Bibliografía

1. S. Bhavsar et al., “Kubernetes Cluster Disaster Recovery Using AWS,” 2023 7th International Conference On Computing, Communication, Control And Automation, ICCUBEA 2023, 2023, doi: 10.1109/ICCUBEA58933.2023.10391973.
2. A. Klos, M. Rosenbaum, and W. Schiffmann, “Scalable and Highly Available Multi-Objective Neural Architecture Search in Bare Metal Kubernetes Cluster,” 2021 IEEE International Parallel and Distributed Processing Symposium Workshops, IPDPSW 2021 - In conjunction with IEEE IPDPS 2021, pp. 605–610, Jun. 2021, doi: 10.1109/IPDPSW52791.2021.00094.
3. G. Budigiri, C. Baumann, J. T. Muhlberg, E. Truyen, and W. Joosen, “Network policies in kubernetes: Performance evaluation and security analysis,” 2021 Joint Eu-

ropean Conference on Networks and Communications and 6G Summit, EuCNC/6G Summit 2021, pp. 407–412, Jun. 2021, doi: 10.1109/EUCNC/6GSUMMIT51104.2021.9482526.

4. A. Goel and B. Thangaraju, “Authenticating Distributed Systems Using SPIRE over Kubernetes Cluster,” 2022 IEEE International Conference on Electronics, Computing and Communication Technologies, CONECCT 2022, 2022, doi: 10.1109/CONECCT55679.2022.9865835.
5. O. Mart, C. Negru, F. Pop, and A. Castiglione, “Observability in Kubernetes Cluster: Automatic Anomalies Detection using Prometheus,” Proceedings - 2020 IEEE 22nd International Conference on High Performance Computing and Communications, IEEE 18th International Conference on Smart City and IEEE 6th International Conference on Data Science and Systems, HPCC-SmartCity-DSS 2020, pp. 565–570, Dec. 2020, doi: 10.1109/HPCC-SMARTCITY-DSS50907.2020.00071.